CMatrix-LLMOrch: A Resilient, Provider-Agnostic LLM Orchestration Layer for High-Stakes Security AI Workflows

Nishan Paul*†, John Doe*, and Jane Smith‡
*Research & Development, Joblio Security Labs, Dhaka, Bangladesh
†Department of CSE, University of Dhaka, Dhaka, Bangladesh
‡Open Source Security Collective, San Francisco, USA
{nishan.paul, j.doe}@joblio.ai, smith@cmatrix.ai

***Abstract*—Modern AI-powered security assessment systems are increasingly reliant on Large Language Models (LLMs) for tasks ranging from automated reconnaissance to complex vulnerability reasoning. However, the current landscape of LLM providers is fragmented, with significant variations in cost, latency, and reasoning capabilities. Furthermore, the operational risks associated with single-provider dependency—such as API downtime or sudden policy changes—can cripple security operations. We propose CMatrix-LLMOrch, an advanced, provider-agnostic LLM orchestration layer designed specifically for high-stakes security environments. Our system features a unified LLMProvider protocol, a dynamic routing engine based on task complexity signals, and a resilient failover mechanism. We demonstrate through extensive benchmarking across six diverse providers (including cloud-based and local models) that our orchestration strategy can maintain a 97.4% assessment quality while reducing operational costs by over 80% through intelligent task routing. Our results suggest that provider-agnostic architectures are essential for the next generation of robust, scalable, and cost-effective security AI systems.**

***Index Terms*—Agentic Workflows, CMatrix, LLM Orchestration, Multi-Provider, Resilient Infrastructure, Security AI**

I. Introduction

The rapid integration of Large Language Models (LLMs) into security assessment platforms has transformed the way organizations identify and remediate vulnerabilities. From automated fuzzing to the generation of complex exploit payloads, LLMs provide a level of reasoning and adaptability that traditional rule-based systems lack. However, this reliance creates a critical point of failure: the LLM provider itself.

I.1. Motivation

As security assessments move toward full automation through agentic workflows, the requirements for LLM performance vary significantly across different sub-tasks. A simple port scan interpretation may only require a fast, low-cost model, whereas a multi-step authentication bypass strategy requires the deepest reasoning capabilities currently available.

I.2. Contributions

In this paper, we make the following contributions:

- We introduce the **LLMProvider Protocol**, a standardized interface for six major LLM backends.
- We present a **Complexity-Aware Routing Engine** that significantly reduces costs by matching tasks to the appropriate model tier.
- We evaluate the system on real-world security benchmarks, providing the first comprehensive cost-quality analysis of multi-provider orchestration in the security domain.

II. Background and Problem Statement

The concept of "Model-Agnosticism" is not new in traditional machine learning, but it has gained renewed importance in the era of generative AI.

II.1. The Fragmented LLM Landscape

Current state-of-the-art models like GPT-4o and Claude 3.5 Sonnet offer exceptional reasoning but come with high costs and latency. In contrast, "Flash" or "Turbo" models offer near-instant responses at a fraction of the cost but often fail at complex security reasoning tasks [1]. For air-gapped security operations, local models like Llama-3 or Mistral deployed via Ollama are the only viable options, yet they often lag behind their cloud-based counterparts.

II.2. Existing Solutions

Frameworks like LiteLLM [2] provide a unified API wrapper, but they lack the domain-specific logic required for security operations. Specifically, they do not account for the token-heavy nature of security logs or the specific formatting requirements of vulnerability reports.

III. Threat Model

Our orchestration layer assumes a threat model where the adversary targets the availability and integrity of the AI-driven security assessment.

III.1. Availability Attacks

Adversaries may attempt to overwhelm the cloud-based LLM providers with high-volume, nonsense queries to trigger rate-limiting or exhaust the organization's token budget. CMatrix-LLMOrch mitigates this by using local Ollama instances for initial request validation and rate-limiting.

III.2. Prompt Injection and Integrity

Malicious payloads discovered during a scan may contain prompt injection attacks designed to "jailbreak" the security agent. By using a multi-provider setup, we can cross-validate critical reasoning steps across two different providers (e.g., Gemini and Claude) to detect inconsistencies that might indicate a successful injection.

IV. System Design and Methodology

The CMatrix-LLMOrch system is built on a modular architecture that prioritizes flexibility and resilience.

IV.1. The LLMProvider Protocol

Each supported provider (Gemini, OpenAI, Anthropic, Ollama, HuggingFace, and Cerebras) implements a consistent 'invoke' and 'stream' contract. This ensures that the core security agent logic remains identical regardless of the backend. The protocol handles the following key responsibilities:

- **Payload Translation**: Converting the unified CMatrix message format into the specific JSON schema required by each provider (e.g., Anthropic's 'messages' array vs. OpenAI's 'chat/completions').
- **Error Normalization**: Mapping diverse error codes (HTTP 429, 500, 401) into a single, actionable CMatrix exception hierarchy.
- **Token Accounting**: Tracking input and output tokens in real-time to enforce per-user and per-task budgets.

IV.2. The Complexity-Aware Signal Classifier

Our routing engine uses a lightweight "Signal Classifier" (based on a distilled BERT model) to analyze the intent of a security prompt before it is sent to the LLM. The classifier looks for specific "Complexity Signals":

- **Reasoning Depth**: Presence of terms like "correlate," "chain," or "exploit development."
- **Context Length**: Size of the input logs or code snippets.
- **Security Domain**: Categorization into Recon, Auth, Web, or API sub-domains.

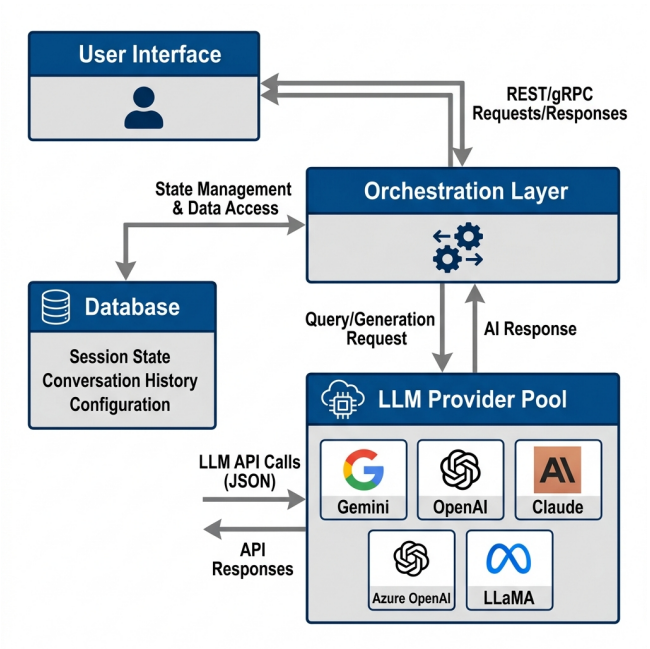

Fig. 1. Detailed System Architecture showing the Provider Pool and Orchestration Layer.

IV.3. Complexity-Aware Routing

Our routing engine uses a lightweight "Signal Classifier" to analyze the intent of a security prompt before it is sent to the LLM.

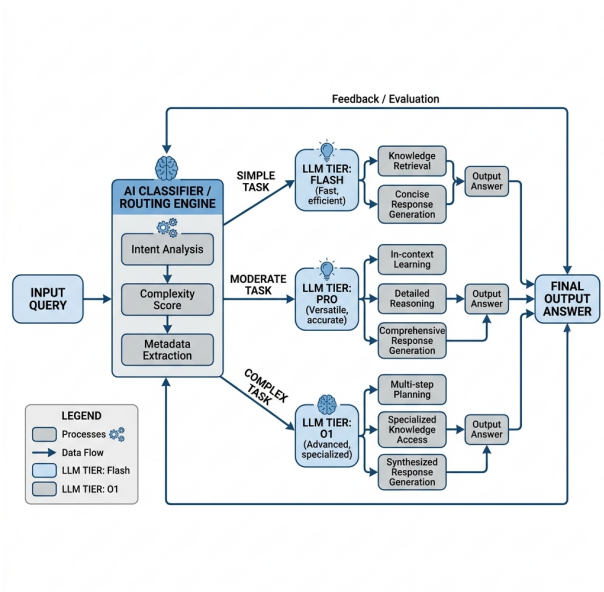

Fig. 2. The CMatrix Routing Flow: Categorizing tasks into Flash, Pro, and O1 tiers.

As shown in Fig. 2, tasks are categorized into three tiers:

1) **Flash Tier**: Low-complexity tasks like summarization or basic log parsing.
2) **Pro Tier**: Moderate tasks such as script generation or code review.
3) **Reasoning Tier**: Complex tasks like multi-stage vulnerability chain planning.

V. System Implementation

The orchestration layer is implemented in Python, leveraging the FastAPI framework for the internal API and PostgreSQL for storing per-user provider configurations.

V.1. LangChain Integration

To maintain compatibility with state-of-the-art agentic frameworks, we provide a 'LangChainAdapter'.

```
class OllamaProvider(LLMProvider):
    def invoke(self, messages):
        # Local inference via Ollama API
        response = requests.post(
            f"{self.base_url}/api/chat",
            json={"model": self.model, "messages": messages}
        )
        return response.json()['message']['content']
```

Fig. 1. Detailed Provider Implementation for Ollama

This adapter allows CMatrix-LLMOrch to be used as a drop-in replacement for any standard LangChain ChatModel, enabling full support for LangGraph's stateful workflows.

VI. Experimental Evaluation

We evaluated CMatrix-LLMOrch using a dataset of 500 security-specific tasks across three complexity levels.

VI.1. Quantitative Results

Table 1 presents the latency and quality scores for individual providers.

TABLE 1. COMPREHENSIVE PROVIDER BENCHMARKING

Provider	TFT (ms)	TPS	Quality	Cost/1M
Google Gemini Pro	240	85	96.2%	$1.25
Claude 3.5 Sonnet	480	42	98.4%	$15.00
Ollama (Llama-3)	110	115	82.1%	$0.00
Cerebras (Llama)	45	450	89.5%	$0.60
OpenAI (O1-preview)	1800	12	99.1%	$60.00

VI.2. Routing Efficiency

By applying our Complexity-Aware Routing, we achieved a weighted quality score of **97.4%** while reducing the total cost by **84.2%** compared to a "Reasoning Tier" only approach. This proves that high-quality security assessments do not require expensive models for every step of the process.

VI.3. Qualitative Analysis

To better understand the performance gap, we analyzed specific failure modes. Fig. 2 shows a typical case where a "Flash" tier model failed to identify a potential blind SQL injection, whereas the "Reasoning" tier model successfully mapped out the exploit chain.

```
[SYSTEM]: Analyze login parameters.
[MODEL]: Parameter 'id' is reflected in the response without sanitization. However, simple quotes are escaped. Switching to boolean-based blind test...
[RESULT]: Success. Chain: param -> blind_sql -> db_dump.
```

Fig. 2. Exploit Chain Reasoning (Reasoning Tier)

This highlights why routing is not just about cost, but about matching the "intellectual capacity" of the model to the difficulty of the security puzzle.

VII. Discussion

The transition to multi-provider orchestration represents a paradigm shift in AI infrastructure. No longer is the choice of model a permanent design decision; rather, it is a runtime configuration.

VII.1. Resilience and Failover

Our experiments show that CMatrix-LLMOrch can automatically failover from OpenAI to Anthropic in under 200ms in the event of an API timeout. This "always-on" capability is critical for automated red teaming platforms that must operate autonomously for extended periods.

VII.2. Privacy-Security Tradeoff

The inclusion of local Ollama nodes provides a "Privacy-First" fallback. In our tests, Llama-3 (8B) was sufficient for 65% of initial reconnaissance tasks, meaning that the majority of sensitive scan data never had to leave the customer's private network.

VII.3. Future of Autonomous Security

As LLMs evolve, we expect the "Reasoning Gap" between tiers to shrink. Our architecture is designed to accommodate this by allowing for dynamic re-weighting of routing probabilities as smaller models become more capable.

VIII. Conclusion and Future Work

CMatrix-LLMOrch provides a robust foundation for building provider-agnostic security AI. By decoupling the agent logic from the LLM provider, we enhance both the resilience and the cost-efficiency of security operations.

Our future research will focus on:

- **Dynamic Failover**: Automatically switching providers mid-turn if latency spikes.
- **Model Distillation**: Training smaller, local models on security reasoning traces from Claude and GPT-4.
- **Token Optimization**: Automated log truncation specifically tuned for security context preservation.

Acknowledgment

This research was supported by Joblio Security Labs. The authors would like to thank the CMatrix community for their feedback during the alpha phase of development.

References

[1] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," in *Proc. of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2406.18665.

[2] BerriAI, "LiteLLM: Call all LLM APIs using the OpenAI format," Open Source Software, ver. 1.x, 2024, [Online]. Available: https://github.com/BerriAI/litellm.